Rapport Projet RL - TP2 AI Driver Unity (PPO)

Ajax DESHAYES--HUET

Introduction

Contexte

Ce TP2 porte sur l'entraînement d'un agent de conduite autonome dans Unity avec ML-Agents, en utilisant l'algorithme PPO. L'agent doit apprendre à suivre la piste de la scène track1 de façon fiable, avec peu de crashes, tout en conservant une progression stable pendant l'entraînement.

Objectifs

Ce rapport a pour objectifs de:

1. présenter la configuration de départ et les hyperparamètres modifiés,
2. comparer les résultats de trois expériences (E1, E2, E3),
3. analyser le compromis entre performance, stabilité et robustesse en inférence,
4. justifier la configuration finale retenue.

Méthodologie

Paramètres modifiables

- Batch size.
- Buffer size.
- Learning rate.
- Epsilon.
- Beta.
- Lambda.

- Num epoch.
- Gamma.

Logique générale du rapport

Le rapport suit une démarche itérative: chaque expérience sert à observer un comportement, identifier une limite, puis ajuster les paramètres.
Les résultats de chaque run orientent directement l'objectif de l'expérience suivante.

Fonction de reward utilisée (non modifiée)

Dans ce TP2, je n'ai pas modifié la fonction de reward entre E1, E2 et E3. Le but était d'isoler l'effet des hyperparamètres PPO (batch size, learning rate, epsilon, beta) sans changer l'objectif d'apprentissage.

Valeurs de reward utilisées dans le script agent (car_agent_template_track1.cs):

- Progression vers la cible: `AddReward(+0.00001)` .
- Éloignement de la cible: `AddReward(-0.001)` .
- Collision avec un objet taggé `Death` : `SetReward(-1.0)` puis fin d'épisode.
- Atteinte d'une cible taggée `Target` : `SetReward(+1.0)` puis passage à la cible suivante.

Ce choix rend la comparaison des trois expériences plus robuste: les différences observées viennent des hyperparamètres PPO, pas d'un reward shaping différent.

Expériences

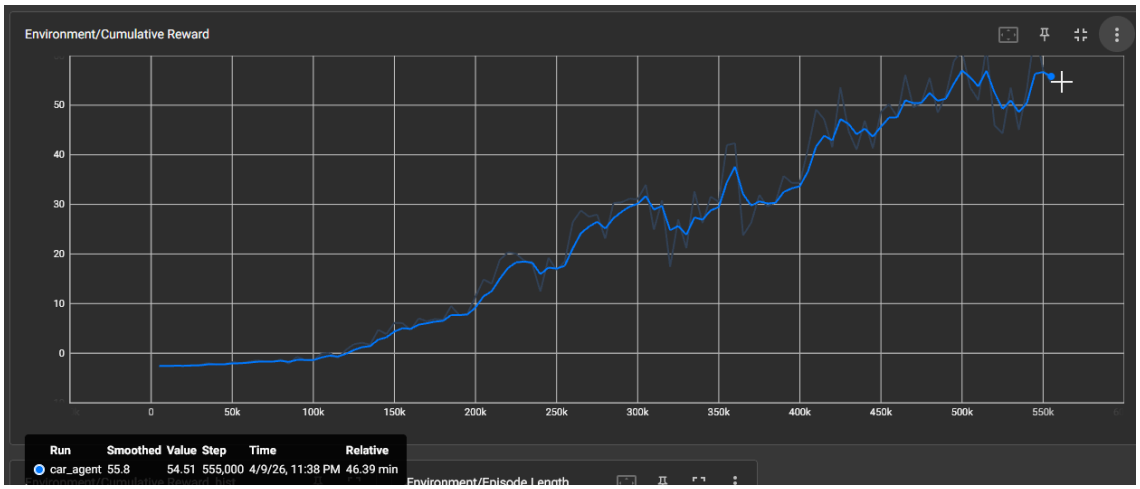
Tableau rapide

Expérience	Batch size	lr	Epsilon	Beta	Lambda	Num epoch	Attente principale
E1	512	3e-4	0.2	0.005	0.95	3	Baseline
E2	1024	1.5e-4	0.15	0.003	0.95	3	Reward finale plus élevée mais moins stable

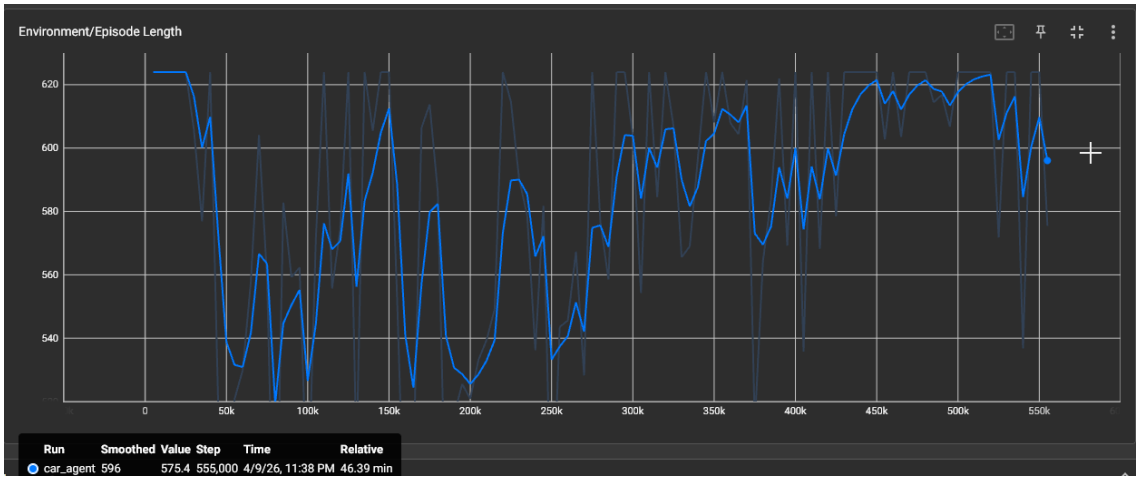
Expérience	Batch size	lr	Epsilon	Beta	Lambda	Num epoch	Attente principale
E3	768	2.2e-4	0.17	0.004	0.95	3	Compromis stabilité-rapidité: apprentissage rapide + inférence robuste

Expérience E1

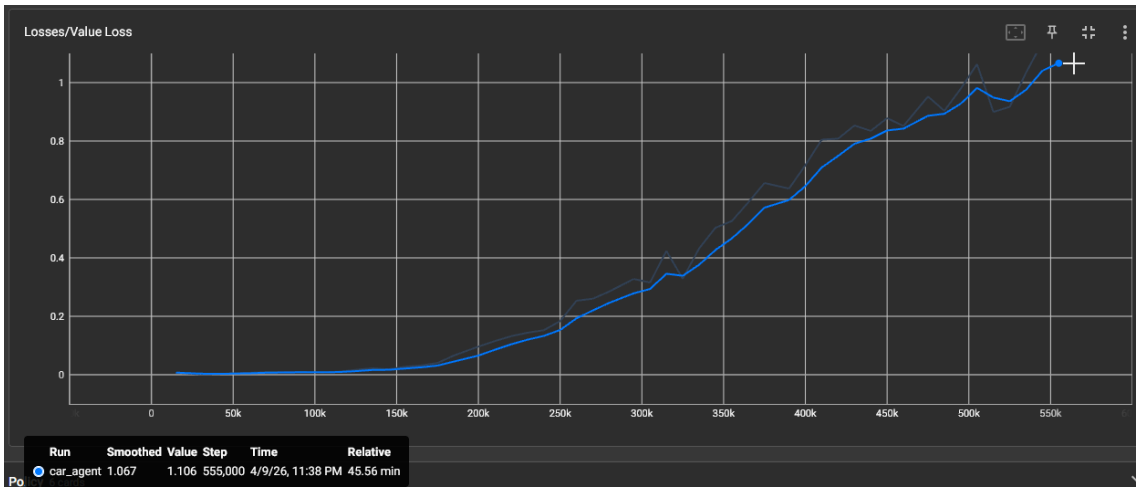
- **Choix des paramètres**: batch size 512, learning rate 3e-4, epsilon 0.2, beta 0.005, lambda 0.95, num epoch 3.
- **Ce qu'on teste**: la baseline PPO sur track1.
- **Résultats observés**:
  - Run lancé avec `--run-id=Experience1`.



- La courbe `Environment/Cumulative Reward` monte globalement tout le long du run. - D'après l'export JSON, la valeur passe d'environ `-2.57` au début à un pic proche de `65.22` (step `545000`), puis termine vers `54.51` (step `555000`) après un dip suivi d'une remontée. -



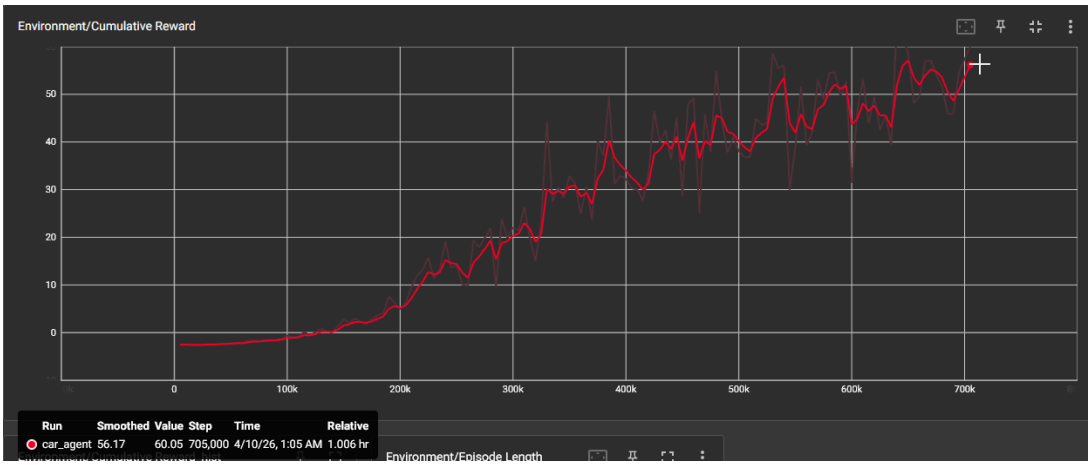
- `Environment/Episode Length` reste bruitée et souvent proche du plafond (`624`), avec quelques baisses ponctuelles (par exemple autour de `453` au step `80000`).



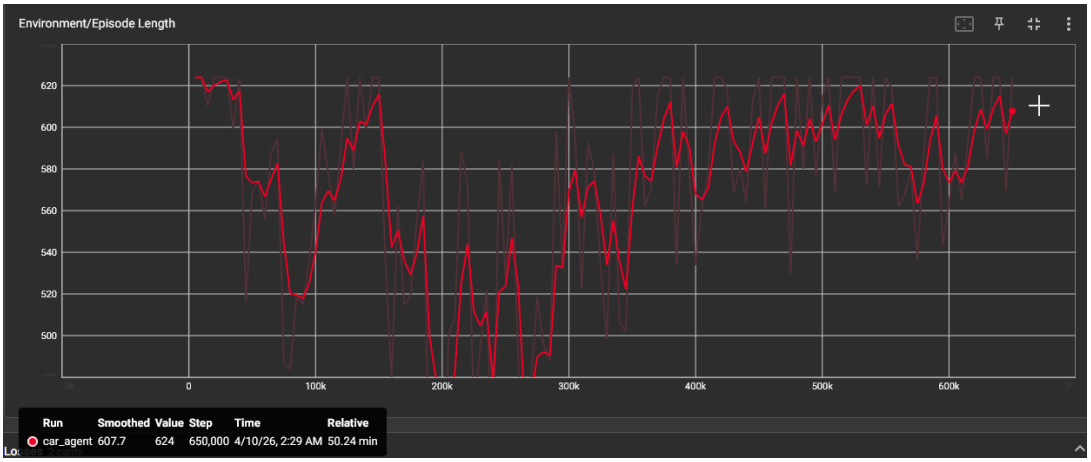
- `Losses/Policy Loss` oscille dans une plage assez stable, globalement entre `0.027` et `0.045`. - `Losses/Value Loss` augmente progressivement (d'environ `0.007` au début jusqu'à ~`1.10` en fin de run), avec des fluctuations. - Test en inférence après entraînement: premier essai en échec (collision mur), deuxième essai validé en `23.26 s` pour un tour. - **Interprétation**: sur ce premier run, j'ai constaté une progression nette de la reward, mais une régularité encore inconstante. La longueur d'épisode souvent élevée et le premier crash en inférence montrent qu'il reste un manque de fiabilité. Le policy loss reste raisonnable, alors que la hausse du value loss suggère que le critic suit plus difficilement quand les retours augmentent. - **Durée du run**: environ `46 minutes`. - **Critère d'arrêt**: j'ai arrêté manuellement après un dip qui s'était rééquilibré, avec une tendance reward encore globalement ascendante.

Expérience E2

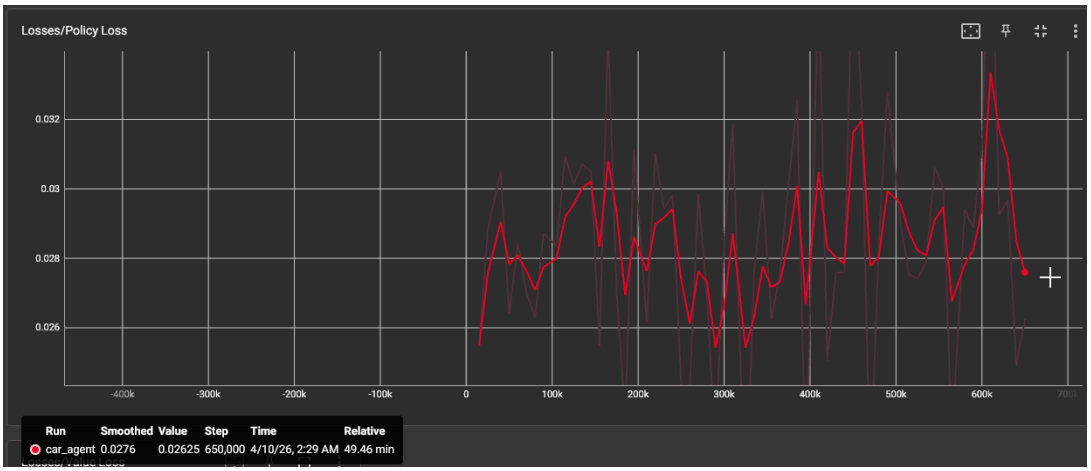
- **Paramètres choisis et justification**:
  - `batch_size` : `512 -> 1024` pour lisser les gradients et limiter les comportements erratiques.
  - `learning_rate` : `3e-4 -> 1.5e-4` pour des updates moins agressives.
  - `epsilon` (PPO clip): `0.2 -> 0.15` pour un clipping plus prudent.
  - `beta` (entropy): `0.005 -> 0.003` pour réduire l'aléatoire en fin d'apprentissage.
  - `lambda` et `num_epoch` inchangés (`0.95` , `3`) pour garder une base comparable à E1.
- **Ce qu'on teste**: conserver la progression reward de E1 avec une conduite plus régulière (moins de collisions, meilleur temps de tour).
- **Résultats observés**:
  - Run lancé avec `--run-id=Experience2` .



- Cumulative reward: -2.53 (début) -> 60.05 (step 705k, **plus élevé que E1 final!**)



- Episode length: similaire à E1, souvent proche du max 624 avec dips ponctuels.

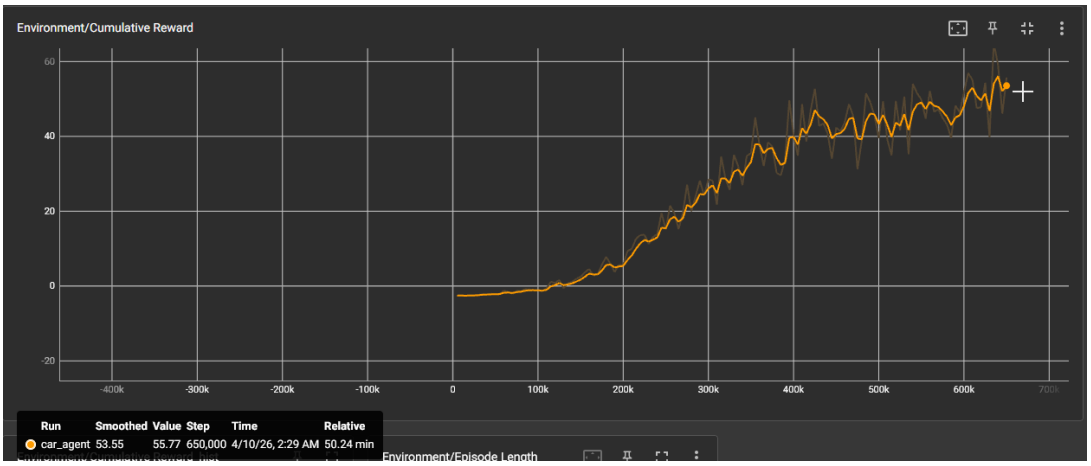


- Policy loss: oscille entre 0.024-0.030 (stable, comparable à E1). - Value loss: monte à 1.47 (légèrement plus instable que E1 qui atteignait max 1.10-1.25). - Tests en inférence: plusieurs crashes et comportements erratiques (agent reste bloqué après avoir raté une reward, crash au début d'autres parcours). Un essai a complété le tour en `20.18 sec` (plus rapide que E1).

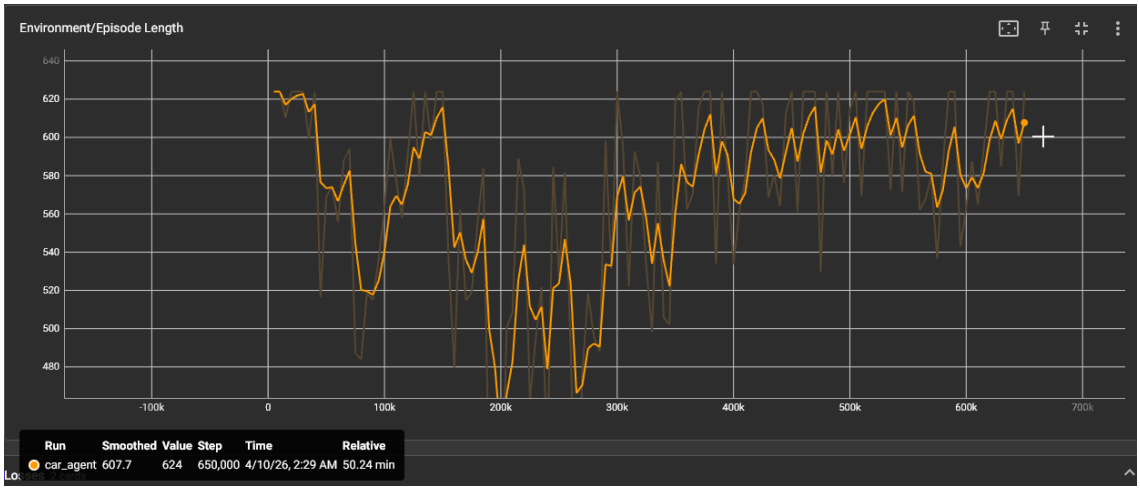
- **Interprétation**: E2 atteint une reward finale plus élevée que E1 (+5.54), avec un tour réussi plus rapide (20.18 s vs 23.26 s). En pratique, j'ai aussi observé plus de runs ratés en inférence (crashes et blocages), ce qui confirme la baisse de fiabilité. Le gain de performance brute est donc réel, mais avec une stabilité plus fragile.

Expérience E3

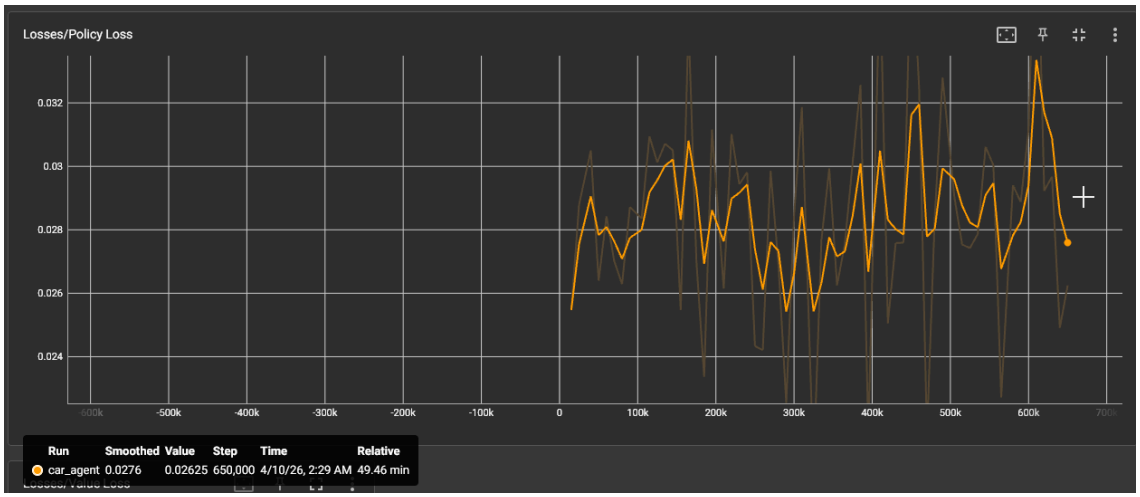
- **Paramètres choisis et justification**:
  - `batch_size` : `E1=512, E2=1024 -> E3=768` pour un compromis de lissage entre E1 et E2.
  - `learning_rate` : `E1=3e-4, E2=1.5e-4 -> E3=2.2e-4` pour équilibrer vitesse d'apprentissage et stabilité.
  - `epsilon` (PPO clip): `E1=0.2, E2=0.15 -> E3=0.17` pour un clipping intermédiaire.
  - `beta` (entropy): `E1=0.005, E2=0.003 -> E3=0.004` pour équilibrer exploration et régularité de conduite.
  - `lambda` et `num_epoch` inchangés (`0.95` , `3`) pour isoler l'effet des autres ajustements.
- **Ce qu'on teste**: un compromis entre E1 et E2, avec une reward finale élevée et une inférence plus robuste.
- **Résultats observés**:
  - Run lancé avec `--run-id=Experience3` .
  - Training duration: environ `50 minutes` pour atteindre 650k steps (`max_steps: 650000` arrêt automatique).

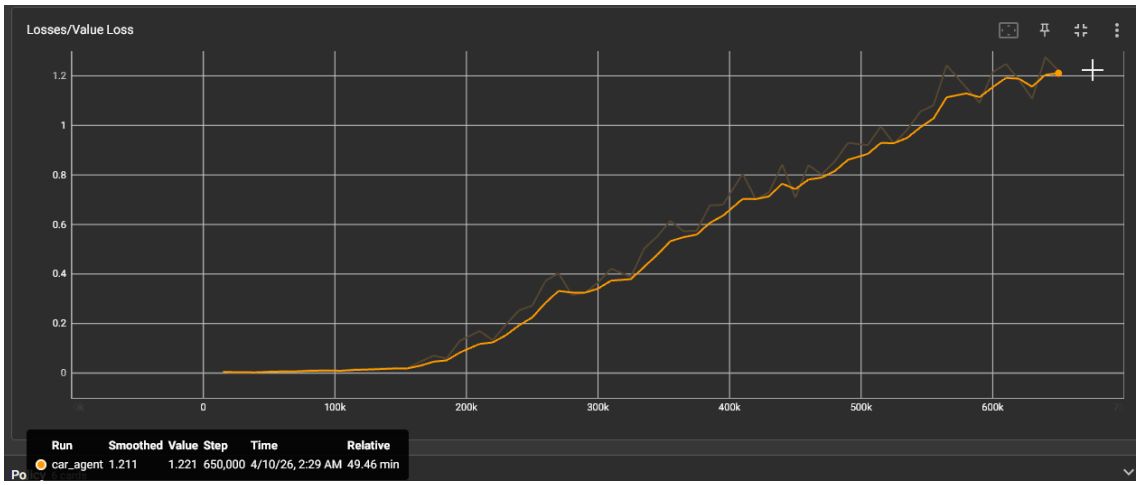


- Cumulative reward: -2.53 (début) -> 55.77 (step 650k)



- Episode length: similaire à E1/E2, souvent proche du max 624 avec dips ponctuels.





- Policy loss: oscille entre 0.025-0.027 (très stable, comparable à E1 et E2). - Value loss: monte à 1.22 (entre E1 ~1.10-1.25 et E2 ~1.47) -> meilleur que E2! - Tests en inférence: agent observé plus stable, rare de rater une reward ou de crash dans les virages serrés. Temps tour mesuré: `20.91 sec`. - **Interprétation**: E3 atteint bien le compromis visé: reward finale 55.77 (entre E1 et E2), value loss à 1.22 (meilleure que E2), et inférence plus fiable que E2. Sur les essais que j'ai faits, la conduite paraissait plus propre dans les virages serrés et moins sujette aux erreurs grossières. Le temps de tour (20.91 s) reste proche de E2 tout en gardant une meilleure stabilité.

Résultats

Synthèse courte: E1 est la baseline la plus stable, E2 atteint la meilleure reward finale mais avec plus d'instabilité, et E3 donne le meilleur compromis global entre performance, stabilité et inférence.

Comparaison finale

Critère	E1	E2	E3
Reward finale	54.51	60.05	55.77
Value loss finale	1.10-1.25	1.47	1.22
Temps tour moyen (réussi)	23.26 s	20.18 s	20.91 s
Robustesse inférence	Stable mais lente	Plus rapide mais moins fiable	Meilleur compromis

Analyse

Les résultats montrent un compromis net entre rapidité et stabilité: E2 maximise la reward, mais perd en robustesse d'inférence. E3 fournit le meilleur équilibre avec une value loss mieux maîtrisée et un comportement plus fiable; les limites restent le faible nombre de runs (1/config) et l'aléa de l'environnement.

Conclusion

Je retiens E3 pour ce TP2: reward finale élevée, value loss mieux contrôlée que E2, et comportement en inférence plus solide. Le point principal du projet reste le même: accélérer l'apprentissage n'a d'intérêt que si la stabilité reste acceptable. Pour la suite, je testerais en priorité un learning rate decay ou une prolongation légère de E3.